ONSITE STORAGE · OSS-NEXT · AGENTIC-AI READINESS

Making the storefront legible to AI agents

Progress report — 34 of 54 tasks complete, 63%.

Date 10 Aug 2026 **Branch** integrate-wp-render **Tracker** `docs/audits/AGENTIC_READINESS.md` **Status** uncommitted working tree

The goal is that AI search crawlers, assistants and shopping agents can find, read and correctly interpret this site. The hard part is structural: ~1,700 of the ~12,000 URLs are WordPress pages rendered as injected Elementor markup, so nothing could be fixed page by page — the fixes had to go into the conversion pipeline and the catch-all route.

34/54	**16**	**4**	**11**	**27**/27
TASKS COMPLETE	REMAINING, BUILDABLE	BLOCKED ON OTHERS	DEFECTS FOUND	AUDIT CHECKS PASS

Progress by phase

PHASE	DONE	STATE
0 · Decisions	4/4	COMPLETE All four answered and recorded
1 · Access & discovery	6/6	COMPLETE
2 · Machine-readable content	6/7	1 BLOCKED needs a Django field
3 · Structured data	8/9	1 BLOCKED needs return-policy terms
4 · Agent API	0/6	NEXT UP unblocked by decision D2
5 · MCP server	0/5	QUEUED requires Phase 4 first
6 · Agentic commerce	0/5	QUEUED scoped by decision D3
7 · Page hygiene	5/6	1 BLOCKED WordPress content work
8 · Measurement & CI	5/6	1 BLOCKED manual, needs live site
Total	**34/54**	**63%**

What is now live

Agents can be admitted, find the site, and read it

- **Crawler policy** — `robots.txt` now names 17 agents individually, each with a recorded reason. Search and user-initiated agents allowed; training-only crawlers blocked.
- **/llms.txt and /llms-full.txt** — a curated index (117 links, all verified) and a full-text variant. Curated, not a sitemap dump: hand-ranked key pages, the depot network, and a stride-sampled set of guides.
- **Markdown view of every page** — `/any-page.md` or `Accept: text/markdown`. The same content at roughly a twentieth of the bytes, with no guesswork about which `<div>` was a heading.

Structured data is complete and interconnected

- Every page emits one `@graph` with a canonical Organization and WebSite at a stable `@id`, referenced rather than restated — so "the seller of this container" resolves to the company described on the homepage.
- Added: BreadcrumbList on PDP and all WordPress pages, FAQPage on the PDP, ItemList on the listing page, LocalBusiness with real service areas on 55 depot pages.
- **Recovered the structured data on ~1,700 WordPress pages** that the conversion pipeline had been silently destroying (see F1).

It is measurable, and regressions get caught

- **Agent traffic logging** into Redis from the proxy, with an admin report at `/admin/agent-traffic`. Costs human traffic nothing.
- **`npm run audit:agentic`** — 27 checks covering schema validity, heading and landmark structure, both Markdown mechanisms, real 404s, robots.txt contents and every llms.txt link. Exits non-zero, so it can gate CI.

Defects found along the way

None of these were on the original task list. They were found by building the checks and then actually running them.

#	DEFECT	RESOLUTION
F1	All ~1,700 WordPress pages shipped **zero structured data** — a `$('script').remove()` added to fix a reCAPTCHA crash was also deleting every `ld+json` block	**FIXED** Schema extracted before the strip. 3 of 11 sampled pages recovered an FAQPage
F2	Missing WordPress paths returned **HTTP 200** with the Not Found page — invisible to any status-code-based link check	**FIXED** Real 404s via a fail-open proxy check. 39/39 real pages verified unaffected
F3	The **monthly rental price** problem: ~8,000 of ~10,000 products carry a monthly figure in `sale_price`. A 40ft container read as "$232.14"	**FIXED** One `getPriceBasis()` feeds Markdown, JSON-LD and meta descriptions
F4	The `/locations` page family **does not exist** — including the Footer link on every page of the site	**FIXED** Repointed at the real depot index
F5	140 depot records were really **55 depots**; 96 are virtual. The naive list showed five city names all linking to Atlanta as separate depots	**FIXED** Deduplicated; virtual entries became the parent's service area
F6	The homepage `SearchAction` pointed at `/products?q=` — a route that has never existed, with a parameter the listing page did not read	**FIXED** Listing search made URL-addressable, then the target restored and verified
F7	The listing page had **no `<h1>` at all**; depot pages had two identical ones	**FIXED** Both, the second in the pipeline so one change covers ~1,700 pages
F8	The listing grid and the PDP spec/FAQ tabs are **client-rendered only** — a fetch of the shop returned zero product names	**FIXED** Server-rendered fallbacks from the same data
F9	`next build` was **already failing** on the branch before this work — a `new Date()` in the Footer	**FIXED** Confirmed pre-existing by building the stashed baseline
F10	The Merchant feed **cannot be prerendered** — it scans ~10,000 documents and takes **~40s**, over the framework's cache timeout	**FIXED** Deferred to runtime. **Note the 40s cold-start cost on deploy**
F11	The new 404 guard **404'd /llms.txt and /llms-full.txt**. The audit reported it as "all 0 links resolve" — a green tick on a broken file	**FIXED** Both the bug and the vacuous pass. Caught by the audit script, before release

Remaining work — 16 tasks we can build

TASK	WHAT
Phase 4 · An API agents can call NEXT	— unblocked by decision D2 (catalog is public, rate-limited)
T4.1	`GET /api/agent/v1/search` — flat JSON, no envelope. Today's only search API speaks InstantSearch's transport shape, which no agent will discover
T4.2	`GET /api/agent/v1/products/{handle}` — full normalised record
T4.3	`GET /api/agent/v1/availability` — can we deliver this container to this zip, and what does delivery cost
T4.4	Hand-written OpenAPI 3.1 spec at `/openapi.json`
T4.5	Rate limiting and caching on the existing Upstash instance
T4.6	Version the API from day one — retrofitting is far worse
Phase 5 · MCP server QUEUED	— thin wrappers over Phase 4, worthless before it
T5.1–5.5	Remote MCP endpoint, four tools mapping 1:1 to the Phase 4 routes, auth posture, connection instructions, tool-call logging
Phase 6 · Agentic commerce QUEUED	— scoped by decision D3 (quote requests, never agent checkout)
T6.1	Fix the Merchant feed's rental pricing the same way the rest of the app now handles it
T6.2	JSON product feed alongside the RSS one — agents parse JSON far more reliably

TASK	WHAT
T6.3	`POST /api/agent/quote` — an agent submits a quote request on a customer's behalf, tagged as agent-originated
T6.4	Explicit price and availability freshness timestamps
T6.5	Agent-facing policy page: what agents may do, rate limits, contact

Blocked — 4 tasks, and no amount of engineering closes them

TASK	OWNER	WHAT IS NEEDED
T3.8	**Sales**	Real return-policy terms — the return window and conditions. Shopping agents check this field before recommending a listing. A guessed policy in structured data is worse than none, so this stays open
T2.4	**Backend team**	A Django-side `agent_summary` field, so the ~1,700 WordPress pages can carry a plain-language summary. The 16 native pages already have one, editable in the admin
T7.3	**Content authors**	Alt text on images inside WordPress-authored markup — 10 of 190 on a sampled depot page. Writing it here would mean inventing descriptions of images we cannot see
T8.5	**Anyone, 15 min/month**	A citation baseline: ask ChatGPT, Claude, Perplexity and Google AI Overviews six fixed buying questions against the live site, record whether we are cited. The question set and results table are ready in the tracker

Verification

✓ `tsc`, `eslint` and `next build` all clean.

✓ `npm run audit:agentic` — 27/27 checks, 117 llms.txt links verified against soft 404s as well as hard ones.

✓ JSON-LD extracted and parsed on every page type; Markdown verified via both suffix and content negotiation.

✓ The 404 change tested against 39 real pages before release — none broken.

One operational item worth flagging before deploy: the Google Merchant feed takes ~40 seconds to build on a cold cache. The deployed function timeout must exceed that, or the first fetch each hour will fail. See F10.